

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Проектування високонавантажених систем»

Лабораторна робота № 2. Лічильник на PostgreSQL: способи оновлення і втрата значень

Виконав: Вовкотруб Олександр Віталійович

Група: ФБ-61мн

Перевірів: Родіонов А. М.

Київ 2026

Зміст

1	Мета роботи	2
2	Постановка задачі	2
3	Хід виконання	3
3.1	Середовище	3
3.2	Схема даних і кластер	3
3.3	Частина I: програма pg-bench	3
3.4	Проба WAL: що коштує кожен варіант на сервері	5
3.5	Частина II: сховище веб-лічильника	6
3.6	Методика вимірювань	6
3.7	Аналітичний ноутбук	7
4	Результати	7
4.1	Частина I: шість способів інкременту	7
4.2	Проба WAL	8
4.3	Відповіді на питання завдання	10
4.3.1	Lost update	10
4.3.2	Serializable: чи буде втрата значень?	10
4.3.3	Serializable: чи будуть помилки?	10
4.3.4	Serializable: як завжди отримувати коректний результат?	10
4.3.5	In-place і SELECT ... FOR UPDATE	10
4.3.6	Optimistic locking	11
4.3.7	Розкид вимірювань	11
4.4	Частина II: порівняння сховищ веб-лічильника	11
5	Додаткове дослідження: чи можна прискорити PostgreSQL	12
6	Висновки	13
7	Джерела	14

1 Мета роботи

Дослідити способи конкурентного оновлення лічильника в СКБД PostgreSQL: з'ясувати, які з них втрачають проміжні значення через стан гонитви, а які дають точний результат, і скільки часу коштує кожен. Використати PostgreSQL як сховище веб-лічильника з лабораторної роботи № 1 і порівняти його продуктивність зі сховищем у пам'яті та на диску.

2 Постановка задачі

Лічильник лайків або переглядів збільшується на одиницю при кожній дії користувача. До нього висуваються дві вимоги [1]: одночасний доступ не повинен призводити до втрати проміжних значень, а дані не повинні зникати після раптового перезавантаження сервера.

Лічильник зберігається в таблиці `user_counter` з полями `user_id` (ключ), `counter` (значення) і `version` (допоміжне поле для оптимістичного блокування). Для одного користувача є один рядок.

Частина I. З 10 окремих клієнтів одночасно збільшити лічильник по 10 000 разів кожним (разом 100 000 інкрементів) і заміряти час для п'яти способів:

1. Lost update: прочитати `counter` через `SELECT`, додати одиницю в клієнті, записати через `UPDATE`.
2. Serializable update: той самий код на рівні ізоляції `SERIALIZABLE`. Відповісти на питання: чи буде втрата значень, чи будуть помилки і чи можна змінити код так, щоб на цьому рівні ізоляції завжди отримувати коректний результат.
3. In-place update: `UPDATE ... SET counter = counter + 1`.
4. Row-level locking: `SELECT ... FOR UPDATE`, потім `UPDATE`. Кожен потік має власне підключення.
5. Optimistic concurrency control: `UPDATE` з умовою на `version` і повтором, доки оновлено нуль рядків.

В усіх варіантах, крім першого, кінцеве значення має дорівнювати 100 000.

Частина II. Використати PostgreSQL з `in-place update` як сховище веб-лічильника з лабораторної роботи № 1, провести тест для 10 клієнтів, що одночасно роблять по 10 000 викликів, і навести таблицю порівняння трьох сценаріїв (у пам'яті, на диску, PostgreSQL) за загальним часом і пропускнуою здатністю.

Обмеження: мова реалізації довільна, ORM заборонені, кожен запис виконується в окремій транзакції.

3 Хід виконання

3.1 Середовище

Клієнти і сервер PostgreSQL працюють на одній машині й спілкуються через loopback. Диск спільний з хост-системою. Параметри наведено в таблиці 3.1.

Таблиця 3.1. Апаратне і програмне середовище

Параметр	Значення
CPU	Intel Coffee Lake (інженерний зразок, 2,60 ГГц), 8 ядер, 16 потоків, частота 0,8...4,4 ГГц
RAM	62 ГіБ (під час вимірювань вільно близько 22 ГіБ)
Диск	NVMe SSD Samsung 970 EVO Plus 1 TB, ext4 (noatime), розділ заповнений на 95 %; на ньому ж дані PostgreSQL
ОС	Fedora Linux 44, ядро 7.2.5-200.fc44.x86_64, без контейнерів
Мова	Swift 6.4, збірка -c release
Драйвер	PostgresNIO (без ORM)
СКБД	PostgreSQL 18.6 з Nix flake, локальний кластер на 127.0.0.1:21410
Налаштування	стандартні: fsync = on, synchronous_commit = on, wal_sync_method = fdatasync, commit_delay = 0, shared_buffers = 128MB, ізоляція за замовчуванням READ COMMITTED

3.2 Схема даних і кластер

Скрипт scripts/pg.sh створює кластер командою initdb у каталозі .pgdata/, запускає й зупиняє його через pg_ctl і застосовує схему з deploy/postgres-init.sql:

```
CREATE TABLE IF NOT EXISTS user_counter (  
    user_id int PRIMARY KEY,  
    counter int NOT NULL,  
    version int NOT NULL  
);  
INSERT INTO user_counter (user_id, counter, version) VALUES (1, 0, 0)  
ON CONFLICT (user_id) DO NOTHING;
```

fsync залишено ввімкненим, тому кожна закомічена транзакція переживає раптове перезавантаження.

3.3 Частина I: програма pg-bench

Програма pg-bench (Sources/pg-bench/) запускає 10 задач у withThrowingTaskGroup. Кожна задача має власне з'єднання PostgreSQL і виконує 10 000 інкрементів. Усі з'єднання

відкриваються, а рядок скидається до (1, 0, 0) до старту годинника, тож вимірюється лише фаза інкрементів:

```
let tally = try await withThrowingTaskGroup(of: Tally.self) { group in
    for connection in connections {
        group.addTask {
            var tally = Tally()
            for _ in 0..
```

Кожен інкремент виконується в окремій явній транзакції BEGIN ... COMMIT (Sources/pg-bench/Transaction.swift). Якщо тіло транзакції або COMMIT завершується помилкою, надсилається ROLLBACK. Ішість способів інкременту описано в Sources/pg-bench/Variant.swift. Варіант serializable з умови реалізовано двічі: без повторів (serializable) і з повтором усієї транзакції (serializable-retry), щоб відповісти на питання завдання вимірюванням.

Lost update під READ COMMITTED читає значення і записує в рядок нове значення, обчислене в клієнті:

```
try await connection.transaction(logger: logger) { connection in
    let counter = try await connection.single(Int32.self,
        "SELECT counter FROM user_counter WHERE user_id = 1",
        logger: logger)
    try await connection.query(
        "UPDATE user_counter SET counter = \(counter + 1) WHERE user_id = 1",
        logger: logger)
}
```

Варіант serializable виконує той самий код після BEGIN ISOLATION LEVEL SERIALIZABLE. Помилка serialization failure (SQLSTATE 40001) рахується як помилка, інкремент відкидається. Варіант serializable-retry повторює транзакцію від BEGIN, доки вона не закомітиться:

```

case .serializableRetry:
  var retries = 0
  while true {
    do {
      try await serializableReadModifyWrite(
        on: connection, logger: logger)
      return .done(retries: retries)
    } catch where error.isSerializationFailure {
      retries += 1
    }
  }
}

```

In-place update виконує один оператор UPDATE user_counter SET counter = counter + 1 WHERE user_id = 1. Варіант for-update додає до SELECT фразу FOR UPDATE, тож блокування рядка береться ще на читанні. Оптимістичне блокування оновлює рядок лише тоді, коли version не змінилась, і починає все спочатку, якщо не повернуто жодного рядка:

```

let updated = try await connection.transaction(logger: logger) { conn in
  let (counter, version) = try await conn.firstRow((Int32, Int32).self,
    "SELECT counter, version FROM user_counter WHERE user_id = 1",
    logger: logger)
  let rows = try await conn.query(
    """
    UPDATE user_counter
    SET counter = \(counter + 1), version = \(version + 1)
    WHERE user_id = 1 AND version = \(version)
    RETURNING version
    """, logger: logger).collect()
  return !rows.isEmpty
}
if updated { return .done(retries: retries) }
retries += 1

```

Як і в псевдокодi умови, невдала спроба завершується COMMIT, а не ROLLBACK.

3.4 Проба WAL: що коштує кожен варіант на сервері

Щоб пояснити різницю в часі даними, скрипт scripts/pg-wal-probe.sh проганяє кожен варіант ще раз у меншому масштабі (10 з'єднань по 1 000 інкрементів) і знімає з сервера дельти лічильників до і після прогону: кількість комітів і відкатів із pg_stat_database та кількість fsync журналу WAL із pg_stat_io [7]:

```
SELECT pg_stat_force_next_flush();
SELECT d.xact_commit, d.xact_rollback,
       (SELECT sum(fsyncs) FROM pg_stat_io WHERE object = 'wal')::bigint
FROM pg_stat_database d WHERE d.datname = 'postgres';
```

Результати пишуться в reports/data/task2-wal-probe.csv і в таблиці часу не входять.

3.5 Частина II: сховище веб-лічильника

Сервер counter-server з лабораторної роботи № 1 отримав сховище PostgresCounterStore (Sources/CounterCore/PostgresCounterStore.swift). Воно тримає пул PostgresClient до 20 з'єднань. GET /inc виконує один in-place UPDATE в autocommit-транзакції, GET /count читає counter:

```
public func increment() async throws {
    try await client.query("""
        UPDATE user_counter SET counter = counter + 1
        WHERE user_id = \(Self.userID)
        """)
}
```

Генератор навантаження loadgen відкриває 10 окремих з'єднань keep-alive. Кожен клієнт надсилає наступний запит лише після відповіді на попередній. Після прогону loadgen перевіряє, що /count дорівнює 100 000.

3.6 Методика вимірювань

Скрипт scripts/task2.sh збирає програми в режимі release і проганяє всі шість варіантів по $10 \times 10\,000$ інкрементів тричі. Кожен запуск записується рядком у reports/data/task2-bench.csv. Частина II той самий скрипт запускає через scripts/task1.sh зі STORES=postgres, також тричі; результати в reports/data/task2-web.csv. Дані для пам'яті й диска взято з reports/data/task1.csv (рядки з 10 клієнтами). У таблицях наведено медіану трьох повторів і діапазон від мінімуму до максимуму.

Перший прогін task2.sh зупинився через ліміт часу фонові задачі (2 год) посеред третього повтору варіанта in-place. Незавершений запуск ніде не записано. Третій повтор для in-place, for-update і optimistic виконано окремим викликом на тому самому кластері й тій самій збірці:

```
PARTS=bench REPEATS=1 VARIANTS="in-place for-update optimistic" \
scripts/task2.sh
```

```
PARTS=web scripts/task2.sh
scripts/pg.sh start && scripts/pg-wal-probe.sh && scripts/pg.sh stop
```

3.7 Аналітичний ноутбук

Обробку даних винесено в ноутбук `solution.ipynb`, опублікований разом зі звітом. Ноутбук читає `task2-bench.csv`, `task2-wal-probe.csv`, `task2-web.csv` і `task1.csv`, рахує в `pandas` медіани й діапазони для таблиць 4.1...4.4, звіряє їх із числами звіту й будує рисунки 4.1...4.3. Розділ «Жива перевірка» при `RUN_LIVE = True` запускає PostgreSQL, проганяє `pg-bench` для `lost-update`, `in-place` і `optimistic` на 10×200 інкрементах і зупиняє сервер. Це демонстрація на зменшеному навантаженні, а не джерело чисел звіту.

4 Результати

4.1 Частина I: шість способів інкременту

Очікуване значення в усіх запусках 100 000. Час і темп наведено в таблиці 4.1, коректність у таблиці 4.2, медіанний час і діапазон на рисунку 4.1.

Таблиця 4.1. Час 100 000 інкрементів (10 з'єднань по 10 000)

Варіант	Час, с (медіана)	Мін., с	Макс., с	Інкр./с (медіана)
lost-update	300,01	166,04	319,15	333,3
serializable	34,55	21,24	38,57	2 894,3
serializable-retry	344,99	287,10	498,35	289,9
in-place	297,92	173,63	307,69	335,7
for-update	324,26	264,01	331,57	308,4
optimistic	1 997,30	1 784,32	2 565,29	50,1

Для `serializable` темп рахується як 100 000 спроб, поділене на час. Успішних інкрементів серед них лише близько 10 000.

Таблиця 4.2. Коректність: мінімум...максимум за три повтори

Варіант	Фінальне значення	Помилки 40001	Повторів на інкремент
lost-update	10 000...10 044	0	0
serializable	10 000...10 003	89 997...90 000	0
serializable-retry	100 000 (3 з 3)	0	8,59...8,79
in-place	100 000 (3 з 3)	0	0
for-update	100 000 (3 з 3)	0	0
optimistic	100 000 (3 з 3)	0	4,89...7,20

Сумарна кількість повторів за три запуски: serializable-retry 879 105, 868 940 і 858 734; optimistic 488 679, 549 821 і 719 674. Фінальні значення lost-update: 10 000, 10 000 і 10 044; serializable: 10 002, 10 000 і 10 003.

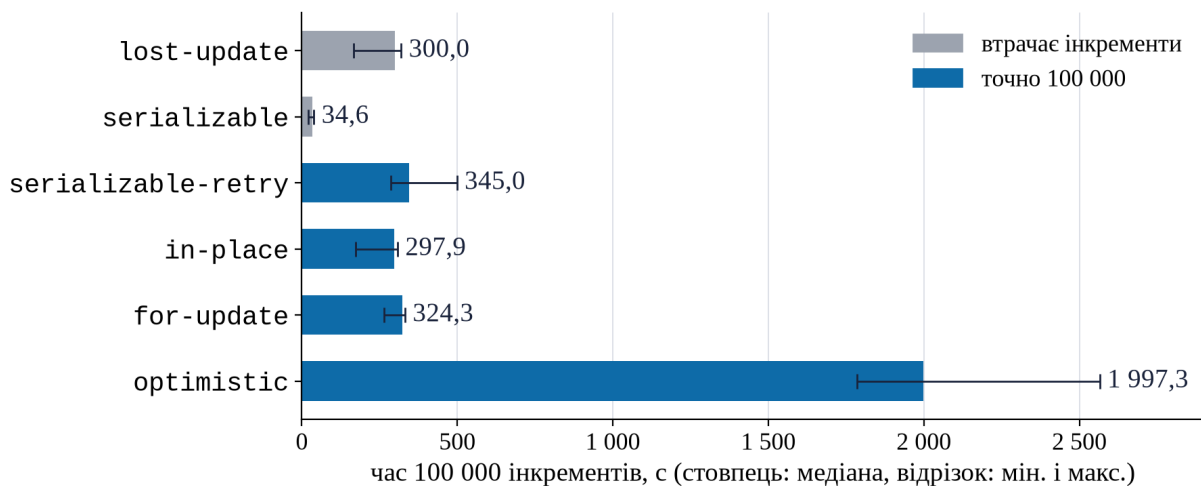


Рисунок 4.1. Час 100 000 інкрементів: медіана трьох повторів (стовпці) і діапазон від мінімуму до максимуму (відрізки); сірим позначено варіанти, що втрачають інкременти

4.2 Проба WAL

Таблиця 4.3 містить результат проби: один прогін $10 \times 1\,000$, тобто 10 000 очікуваних інкрементів. Кілька зайвих комітів дають службові запити pg-bench і скрипта (скидання рядка, читання результату, знімки статистики).

Таблиця 4.3. Коміти, відкати і fsync WAL на 10 000 інкрементів

Варіант	Час, с	COMMIT	ROLLBACK	fsync WAL	fsync на інкр.
lost-update	26,91	10 014	0	10 003	1,000
serializable	2,73	1 022	8 992	1 017	0,102
serializable-retry	26,25	10 019	83 866	10 052	1,005
in-place	16,86	10 014	0	10 001	1,000
for-update	19,53	10 014	0	10 001	1,000
optimistic	186,98	59 904	0	59 884	5,988

У цьому прогоні serializable дав фінальне значення 1 008 при 8 992 помилках, serializable-retry зробив 83 866 повторів, optimistic 49 875 повторів; решта варіантів, крім lost-update (1 000), дали точно 10 000.

Проба показує, що час визначає кількість записуючих комітів. Із fsync = on кожен COMMIT транзакції, що щось записала, чекає скидання WAL на диск. ROLLBACK диска не чекає. Усі варіанти з одним записуючим комітом на інкремент (lost-update, serializable-retry, in-place, for-update) мають рівно один fsync на інкремент. У serializable 9 з 10 транзакцій відкочуються, тому fsync лише 0,102 на спробу. У optimistic кожна невдала спроба теж комітить і чекає диск: близько 6 fsync на інкремент.

На рисунку 4.2 ці дані поставлено поруч із кількістю повторів на інкремент з основного вимірювання (таблиця 4.2). Повтори serializable-retry закінчуються ROLLBACK і fsync не додають, а повтори optimistic закінчуються COMMIT і додають.

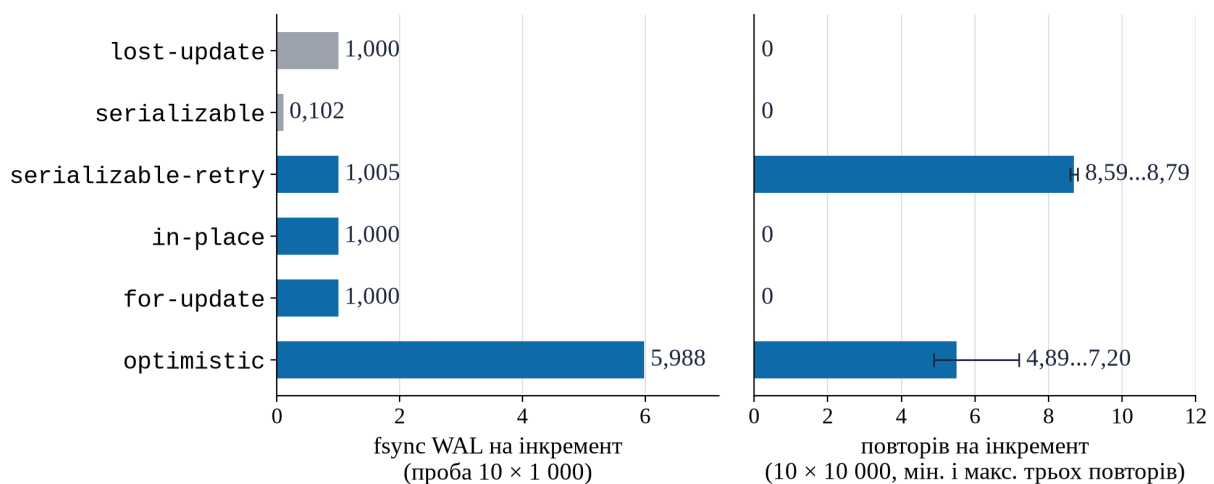


Рисунок 4.2. fsync WAL на інкремент у пробі 10 × 1 000 (ліворуч) і повтори на інкремент у вимірюванні 10 × 10 000, медіана й діапазон трьох повторів (праворуч)

4.3 Відповіді на питання завдання

4.3.1 Lost update

Фінальне значення близько 10 000 замість 100 000: втрачено близько 90 % інкрементів, і жодна операція не повідомила про помилку. SELECT під READ COMMITTED не блокується, тому всі 10 з'єднань читають одне закомічене значення v , поки одне з них тримає блокування рядка й чекає `fsync` при COMMIT. Потім усі десять по черзі записують те саме $v + 1$. Кожен «раунд» з 10 транзакцій додає лише одиницю. При цьому кожна з 100 000 транзакцій пише й комітить (1,000 `fsync` на інкремент), тому за часом цей варіант не дешевший за коректний `in-place` (300,01 с проти 297,92 с).

4.3.2 Serializable: чи буде втрата значень?

Закомічені інкременти не втрачаються. Фінальне значення точно дорівнює кількості успішних транзакцій: $100\,000 - 89\,998 = 10\,002$, $100\,000 - 90\,000 = 10\,000$, $100\,000 - 89\,997 = 10\,003$, і це збігається з фінальним значенням у всіх трьох повторях. Рівень SERIALIZABLE не допускає lost update [2], але близько 90 % інкрементів не виконано взагалі.

4.3.3 Serializable: чи будуть помилки?

Так. Від 89 997 до 90 000 транзакцій зі 100 000 завершилися помилкою `serialization failure` (SQLSTATE 40001). Механізм той самий, що в lost update: 10 транзакцій читають один знімок, перша, що закомітила UPDATE, виграє, а решта при спробі оновити рядок, змінений після їхнього знімка, отримують 40001 і відкочуються. Виходить приблизно один успіх на десять спроб. Варіант найшвидший (34,55 с) лише тому, що 9 з 10 транзакцій закінчуються ROLLBACK без очікування диска.

4.3.4 Serializable: як завжди отримувати коректний результат?

Транзакцію, що впала з 40001, треба повторити повністю, від BEGIN, включно з SELECT, бо прочитане значення застаріло. Повтор робиться, доки транзакція не закомітиться. PostgreSQL прямо рекомендує такий підхід для рівня SERIALIZABLE [2]. Це варіант `serializable-retry`: у всіх трьох повторях фінальне значення рівно 100 000 і помилок немає. Ціна становить від 858 734 до 879 105 повторів, тобто близько 8,7 відкинутої спроби на кожен інкремент. Відкинуті спроби закінчуються ROLLBACK і `fsync` не потребують (1,005 `fsync` на інкремент), тому медіанний час 344,99 с лише на 16 % більший, ніж у `in-place` (297,92 с). Зайва робота припадає на мережеві round-trip і CPU, а не на диск.

4.3.5 In-place і SELECT ... FOR UPDATE

Обидва варіанти точні в усіх повторях, без помилок і повторів. UPDATE ... SET counter = counter + 1 читає й пише під блокуванням рядка, а FOR UPDATE бере це блокування ще на читанні [3]. Блокування тримається до COMMIT, а COMMIT чекає `fsync` WAL, тому всі 100 000

транзакцій ідуть строго послідовно: $297,92 \text{ с} / 100\,000 \approx 3,0 \text{ мс}$ на транзакцію. Це і є затримка `fdatasync WAL` на цьому диску. `for-update` трохи повільніший (324,26 с), бо блокування тримається на один `round-trip` довше: два оператори замість одного.

4.3.6 Optimistic locking

Варіант точний, але найповільніший: медіана 1 997,30 с, 50,1 інкр./с, від 4,9 до 7,2 повтору на інкремент. Невдала спроба (оновлено нуль рядків) завершується `COMMIT`, і цей `COMMIT` теж чекає `fsync`: 59 904 коміти й 59 884 `fsync` на 10 000 інкрементів у пробі. PostgreSQL виділяє транзакції ідентифікатор уже на спробі `UPDATE`, тому навіть «порожній» коміт пише запис у `WAL`. Крім того, конкурентний `UPDATE ... WHERE version = v` під `READ COMMITTED` спершу чекає, поки переможець закомітить, і лише потім бачить, що версія змінилась [3]. Невдала спроба коштує очікування і чужого `fsync`, і свого. Оцінка $1\,997,30 \text{ с} / (100\,000 \cdot 6,5 \text{ коміту}) \approx 3,1 \text{ мс}$ на коміт збігається із затримкою `fsync` у `in-place`. Оптимістичне блокування добре працює при рідких конфліктах [4], а на одному «гарячому» рядку з 10 записувачами конфлікт майже гарантований.

4.3.7 Розкид вимірювань

У кожного варіанта, обмеженого `fsync`, один із трьох запусків помітно швидший: `lost-update` 166,04 с, `in-place` 173,63 с, `for-update` 264,01 с, веб-лічильник на PostgreSQL 199,76 с. Такий самий дворівневий режим затримки `fsync` спостерігався в лабораторній роботі № 1 і пояснювався фоновією активністю хоста на спільному, майже заповненому диску. Медіани відповідають повільному режиму; на висновки розкид не впливає.

4.4 Частина II: порівняння сховищ веб-лічильника

Навантаження: 10 клієнтів по 10 000 запитів `GET /inc`, три повтори. Фінальне значення в усіх дев'яти запусках рівно 100 000. Результати наведено в таблиці 4.4 і на рисунку 4.3.

Таблиця 4.4. Загальний час і пропускна здатність веб-лічильника (медіани трьох повторів)

Сховище	Загальний		Пропускна	
	час, с	здатність, RPS	RPS, діапазон	Коректно
In-memory (<code>Atomic<Int></code>)	3,20	31 256	30 215...34 300	3 з 3
Disk (файл і <code>fsync</code>)	818,08	122,2	121,3...122,8	3 з 3
PostgreSQL (<code>in-place UPDATE</code>)	302,44	330,6	326,6...500,6	3 з 3

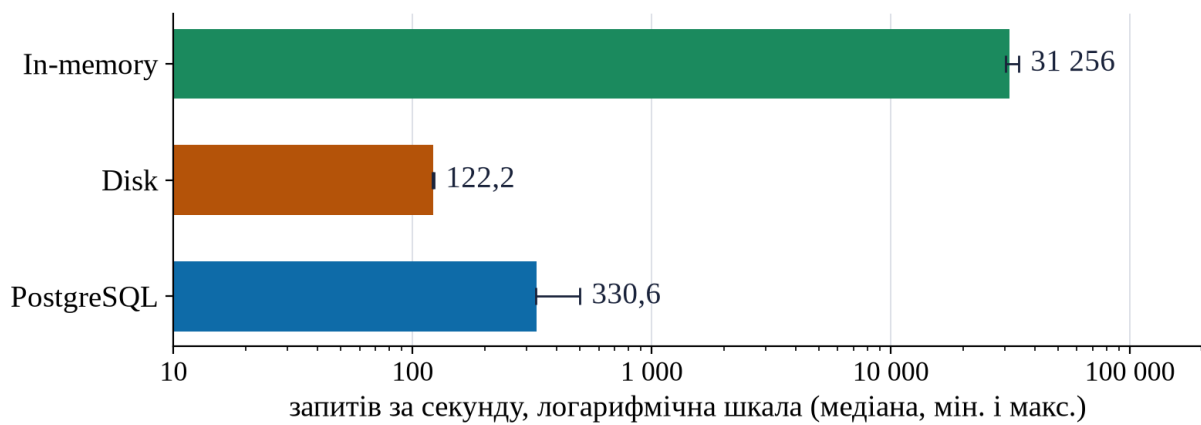


Рисунок 4.3. Пропускна здатність веб-лічильника за сховищами, логарифмічна шкала: медіана трьох повторів і діапазон від мінімуму до максимуму

Веб-лічильник на PostgreSQL (330,6 RPS) майже збігається з `pg-bench in-place` (335,7 інкр./с): накладні витрати HTTP і пулу з'єднань на фоні 3 мс коміту непомітні. Він у 2,7 раза швидший за файловий лічильник (122,2 RPS). Обидва чекають диск на кожен інкремент, але PostgreSQL дописує послідовний журнал WAL через `fdatasync`, а файловий лічильник перезаписує файл і робить повний `fsync`. Порівняно з лічильником у пам'яті (31 256 RPS) PostgreSQL повільніший приблизно в 95 разів. Це ціна гарантії, що кожен підтверджений інкремент переживе збій.

5 Додаткове дослідження: чи можна прискорити PostgreSQL

Окремо перевірено, чи можна підняти пропускну здатність веб-лічильника на PostgreSQL, не порушуючи правил завдання (окрема транзакція на запис, `fsync = on`, `synchronous_commit = on`). Вимірювання йшли почергово (базова версія, нова, базова, нова), по $10 \times 2\,000$ запитів, три повтори, медіани; сирі дані в `reports/data/throughput-ab.csv`. Затримка `fsync` на цьому SSD має два стабільні рівні, тому абсолютні значення між серіями можуть різнитися приблизно вдвічі; порівнювати варто лише варіанти всередині однієї серії.

Вузьке місце пояснює механізм коміту. Блокування рядка відпускається лише після того, як `COMMIT` дочекався скидання WAL. Наступна транзакція на тому ж рядку стартує після чужого `fdatasync`, тому об'єднати коміти в одне скидання (`group commit`) неможливо. Параметр `commit_siblings` не враховує бекенди, що чекають на блокування (функція `MinimumActiveBackends` у `procarray.c` [9]), тож `commit_delay` [5] тут лише додає затримку.

Таблиця 5.1. Розмір пулу і рознесення лічильника на рядки (10 × 2 000)

Варіант	RPS (медіана)	RPS, діапазон
базовий, пул 20 з'єднань	662,7	652,2...677,6
пул 10 з'єднань	655,8	585,2...673,9
пул 1 з'єднання	572,2	305,6...598,2
postgres-sharded, 10 рядків	3 343,0	1 761,1...3 560,0

Таблиця 5.2. Налаштування сервера (commit_siblings = 1, 10 × 2 000)

Налаштування	Один рядок, RPS	10 рядків, RPS
commit_delay = 0, fdatsync (стандарт)	659,8	3 240,9
commit_delay = 0, open_datasync	670,9	3 412,2
commit_delay = 200	576,2	2 945,9
commit_delay = 1000	267,0	2 058,1

Для одного рядка розмір пулу нічого не дає: 10 і 20 з'єднань працюють однаково, а одне з'єднання трохи гірше, бо до затримки коміту додається очікування в черзі клієнта. open_datasync дає те саме, що fdatsync, у межах шуму [6]. commit_delay шкодить в обох схемах: на одному рядку групувати нічого, а на 10 рядках групування відбувається й без нього. Після серії всі параметри скинуто до стандартних.

Сховище postgres-sharded реалізовано в каталозі Sources/CounterCore/, файл PostgresShardedCounterStore.swift. Воно розкладає лічильник користувача на 10 рядків таблиці user_counter_shard, а значення рахує як SUM(counter). Кожен /inc лишається одним inplace UPDATE в окремій autocommit-транзакції. Сусідні запити йдуть у різні рядки по колу, тож до 10 транзакцій тримають блокування одночасно, і PostgreSQL комітить їх одним скиданням WAL [8]. Виграш у серії становить ×5,0, а на повному сценарії 10 × 10 000 один прогін дав 3 444,4 RPS проти 330,6 RPS у частині II; /count дорівнював 100 000.

Це межовий випадок: жоден запис не втрачено і кожен переживе перезавантаження, але змінено модель даних, а завдання описує один рядок user_counter на користувача. Оригінальне сховище postgres не змінено.

6 Висновки

Частина I.

1. Наївний read-modify-write під READ COMMITTED (lost-update) втрачає близько 90 % оновлень: фінальне значення від 10 000 до 10 044 зі 100 000, без жодної помилки.
2. SERIALIZABLE не дає втратити закомічені оновлення: фінальне значення завжди дорівнює кількості успішних транзакцій. Без повторів близько 90 % транзакцій падають

- з 40001. Повтор усієї транзакції від BEGIN при 40001 дає рівно 100 000 ціною близько 8,7 повтору на інкремент і 16 % часу понад in-place (344,99 с проти 297,92 с).
3. Найпростіший і найшвидший серед коректних способів це атомарний UPDATE ... SET counter = counter + 1: медіана 297,92 с, 335,7 інкр./с. SELECT ... FOR UPDATE майже такий самий (324,26 с). Оптимістичне блокування на одному гарячому рядку найгірше (1 997,30 с), бо кожна невдала спроба теж комітить і чекає fsync, близько 6 fsync на інкремент.
 4. Пропускна здатність усіх коректних варіантів обмежують затримка fsync WAL (близько 3 мс на коміт) і серіалізація на блокуванні одного рядка, а не CPU чи мережа. Проба WAL підтверджує рівно один fsync на інкремент для in-place, for-update і serializable-retry.

Частина II.

5. Як сховище веб-лічильника PostgreSQL дає 330,6 RPS при 10 клієнтах (302,44 с на 100 000 запитів): у 2,7 рази більше, ніж файл з fsync (122,2 RPS), і приблизно в 95 разів менше, ніж пам'ять (31 256 RPS). Усі 100 000 інкрементів збережено в кожному запуску.

Додаткове дослідження.

6. Для одного рядка ні розмір пулу, ні wal_sync_method, ні commit_delay пропускну здатність не підвищують, а commit_delay = 1000 знижує її з 659,8 до 267,0 RPS. Рознесення лічильника на 10 рядків вмикає group commit і дає $\times 5,0$ (3 444,4 RPS на $10 \times 10\,000$), але змінює модель даних, тому подано як межовий варіант.

7 Джерела

1. Родіонов А. М. Task 2. Реалізація каунтера з використанням PostgreSQL : завдання до лабораторної роботи [Електронний ресурс]. Режим доступу: <https://docs.google.com/document/d/1gDPSHPDchLJbwhNaNzjiPIch14n-0JuHnFxkyeghcwk> (дата звернення: 05.10.2026).
2. PostgreSQL Documentation. Transaction Isolation [Електронний ресурс]. Режим доступу: <https://www.postgresql.org/docs/current/transaction-iso.html> (дата звернення: 05.10.2026).
3. PostgreSQL Documentation. Explicit Locking [Електронний ресурс]. Режим доступу: <https://www.postgresql.org/docs/current/explicit-locking.html> (дата звернення: 05.10.2026).
4. PostgreSQL Documentation. Data Consistency Checks at the Application Level [Електронний ресурс]. Режим доступу: <https://www.postgresql.org/docs/current/applevel-consistency.html> (дата звернення: 05.10.2026).
5. PostgreSQL Documentation. WAL Configuration [Електронний ресурс]. Режим доступу: <https://www.postgresql.org/docs/current/wal-configuration.html> (дата звернення: 05.10.2026).

6. PostgreSQL Documentation. Write Ahead Log [Електронний ресурс]. Режим доступу: <https://www.postgresql.org/docs/current/runtime-config-wal.html> (дата звернення: 05.10.2026).
7. PostgreSQL Documentation. The Cumulative Statistics System [Електронний ресурс]. Режим доступу: <https://www.postgresql.org/docs/current/monitoring-stats.html> (дата звернення: 05.10.2026).
8. Geoghegan P. Towards 14,000 write transactions per second on my laptop [Електронний ресурс]. Режим доступу: <http://pgeoghegan.blogspot.com/2012/06/towards-14000-write-transactions-on-my.html> (дата звернення: 05.10.2026).
9. PostgreSQL Global Development Group. procarray.c : вихідний код PostgreSQL [Електронний ресурс]. Режим доступу: <https://github.com/postgres/postgres/blob/master/src/backend/storage/ipc/procarray.c> (дата звернення: 05.10.2026).
10. PostgresNIO: non-blocking, event-driven Swift client for PostgreSQL [Електронний ресурс]. Режим доступу: <https://github.com/vapor/postgres-nio> (дата звернення: 05.10.2026).